

Post-Generation Tool Governance for Autonomous AI Agents: A Deterministic Reference-Monitor Architecture and Empirical Evaluation

Shriram Baskar

Independent Researcher

https://github.com/Shriram28Baskar/PromptAegis_RI

Abstract—Autonomous language model (LLM) agents interacting with external digital environments inherit significant security risks from indirect prompt injection and context poisoning. While prior research predominantly investigates pre-generation alignment and probabilistic token filtering, these techniques cannot enforce deterministic execution invariants once an attacker manipulates the model's internal reasoning context.

This paper investigates whether deterministic, post-generation execution governance can constrain the security consequences of compromised tool calls while preserving legitimate operational utility and maintaining measurable execution latency. We present **PromptAegis**, an application-level reference monitor positioned strictly between an agent's structured action proposal and the concrete tool execution dispatcher. PromptAegis mediates routed tool invocations through a sequential five-stage pipeline: volumetric fixed tumbling-window rate limiting, deterministic role-based access control (RBAC), argument-level policy validation incorporating keyword-agnostic parameter canonicalization, risk scoring with approval gating, and structured audit logging.

We evaluate PromptAegis across a primary synthetic benchmark of $N = 500$ attack scenarios and $N = 100$ benign operational commands. To eliminate confounding between semantic policy inspection and volumetric rate saturation, we implement a mechanism-isolation ablation protocol across six configurations (3,000 attack evaluations and 3,600 total scenario decisions). Under isolated semantic evaluation with rate limiting inactive, PromptAegis reduces the Attack Success Rate (ASR) from 100.0% (500/500) in the unprotected baseline to 40.0% (200/500) under RBAC alone, 33.0% (165/500) under policy inspection alone, and 30.0% (150/500) under combined normalized inspection (McNemar Edwards $\chi^2 = 348.0029, p = 1.15 \times 10^{-77}$). In a calibrated gateway configuration (incorporating refined argument schemas and tool-scoped parameter fallback rules), residual ASR is reduced to 20.4% (102/500) on the evaluated benchmark while maintaining a 100.0% (100/100) Legitimate Task Completion Rate (LTCR) and 0.0% (0/100) False Positive Rate (FPR). The median paired per-scenario latency overhead is +11.27 ms under normalized traffic and +13.38 ms under burst traffic.

Adversarial stress testing across 500 mutation instances drawn from a clustered perturbation design (5 classes $\times$ 100 base templates; 2,000 execution evaluations) exposes critical non-monotonic trade-offs: while structural Base64 canonicalization increases isolated evasion block rates from 0.0% (0/100) to 50.0% (50/100), overall isolated block rate decreases from 40.0% (200/500) to 36.4% (182/500) because aggressive comment stripping alters string character offsets for secondary regex rules. In compound burst testing, the standard pipeline blocks 75.2% (376/500, 24.8% ASR); however, mechanism attribution confirms that 95.74% (360/376) of these blocks are driven by rate-limiter saturation rather than parameter inspection. In an exploratory closed-loop pilot ($N = 20$) with a live commercial LLM (gpt-oss-120b), PromptAegis intercepted 75.0% (3/4) of rogue tool calls emitted following successful prompt injections, limiting unintercepted hostile tool execution to 10.0% (1/10).

Our findings indicate that deterministic post-generation governance provides measurable execution mitigation against unauthorized actions, but its protection remains strictly bounded by syntactic policy expressiveness, cannot prevent parameter smuggling within authorized semantic spaces, and does not alter the underlying vulnerability of the model's reasoning.

Keywords—Large Language Model Security, Autonomous AI Agents, Indirect Prompt Injection, Execution Governance, Reference Monitor, Access Control, Parameter Canonicalization.

1. INTRODUCTION

The integration of Large Language Models (LLMs) into autonomous agentic workflows—such as ReAct [1], Toolformer [2], and AutoGen [3]—enables generative models to issue executable actions directly against shell environments, databases, and external APIs. This paradigm shift introduces a fundamental challenge in computer systems security: whereas classical security engineering dictates that untrusted input must never determine control flow or execution privileges [4], autonomous agents ingest untrusted external context (e.g., retrieved web text, email content, ticketing data) into the prompt context that directs execution planning. Consequently, adversaries can stage *indirect prompt injection* attacks [6], manipulating the model's internal reasoning into emitting unauthorized, high-privilege, or destructive tool invocations.

Existing defensive strategies focus primarily on **pre-generation alignment** (e.g., RLHF [7]) and **system prompt instructions**, though empirical evaluations demonstrate that natural language instructions fail to reliably prevent prompt injection [8], alongside **probabilistic token guardrails** (e.g., Llama Guard [9], NeMo Guardrails [10]). While these techniques raise the cost of basic

jailbreaks, they treat security as a statistical property of natural language tokens. Because generative models operate over unsegmented natural language, probabilistic filtering cannot guarantee deterministic execution invariants against out-of-distribution adversarial prompts [11].

This paper investigates an alternative, complementary principle: **separating reasoning security from execution governance**. An autonomous agent's reasoning engine may be manipulated by adversarial text, but catastrophic outcomes can be prevented if the concrete tool invocations emitted by the model are mediated by an external, application-level *reference monitor* [12, 13]. Rather than attempting to force the LLM into a provably safe cognitive state, security architectures can treat the model as an untrusted computation engine operating within an adversary's potential sphere of influence.

We design, implement, and evaluate **PromptAegis**, an application-level, deterministic post-generation tool governance gateway. Positioned strictly between the LLM output parser and the concrete tool execution dispatcher, PromptAegis mediates every proposed function call through a multi-stage deterministic pipeline: 60-second fixed tumbling-window rate limiting, role-based access control (RBAC), argument-level policy inspection incorporating

keyword-agnostic parameter canonicalization, risk scoring with approval gating, and audit telemetry.

1.1 Research Questions

- **RQ1 (Semantic Efficacy):** To what extent can deterministic RBAC and argument-level regex policies reduce the Attack Success Rate (ASR) of compromised tool calls in the absence of rate-limiting confounding?
- **RQ2 (Utility Preservation & False Positives):** What Legitimate Task Completion Rate (LTCR) and False Positive Rate (FPR) are observed on the benign operational corpus when post-generation policy inspection is applied?
- **RQ3 (Latency Overhead):** What execution latency overhead does post-generation mediation introduce relative to unmediated tool dispatch?
- **RQ4 (Adversarial Evasion & Trade-offs):** How do parameter canonicalization rules perform against obfuscated payloads, and does normalization introduce negative performance trade-offs?
- **RQ5 (Dynamic Closed-Loop Feasibility):** In an end-to-end agent loop with a live LLM, what proportion of prompt-injection-induced tool calls are intercepted post-generation?

2. PROBLEM FORMULATION

Let an autonomous agent $\mathcal{A}$ operate over discrete interaction steps $t \in \{1, \dots, T\}$. At step t , the agent receives an environment observation $o_t \in \mathcal{O}$ and interaction history $h_t = (u, o_1, a_1, \dots, o_{t-1}, a_{t-1})$, where $u \in \mathcal{U}$ is the primary user task instruction. Observation o_t may contain untrusted data from external sources. The agent's autoregressive policy $\mathcal{M}_\theta$ emits a token sequence parsed into a structured action proposal:

$$a_t = (r, \tau, \theta_\tau)$$

where $r \in \mathcal{R}$ is the active agent role, $\tau \in \mathcal{T}$ is the target tool identifier, and $\theta_\tau \in \Theta_\tau$ is the dictionary of structured arguments.

In an unmitigated architecture, tool dispatch is direct:

$$\mathcal{E}_{t+1} \leftarrow \text{Exec}(\tau, \theta_\tau)$$

When an adversary embeds an injection payload δ_{adv} within o_t , the conditional probability of emitting a malicious tool call $a_t \in \mathcal{A}_{\text{malicious}}$ increases. PromptAegis introduces a deterministic reference monitor function $\mathcal{G} : \mathcal{A} \times \mathcal{S}_{\text{state}} \rightarrow \{\text{ALLOW}, \text{BLOCK}, \text{REQUIRE_APPROVAL}\}$ such that:

$$\mathcal{G}(a_t) = \begin{cases} \text{ALLOW} & \implies \text{Exec}(\tau, \theta_\tau) \\ \text{BLOCK} & \implies \text{Reject}(\epsilon_\tau) \\ \text{REQUIRE_APPROVAL} & \implies \text{Halt}(\epsilon_\tau) \end{cases}$$

The operational objective is to minimize ASR ($\text{ASR} \rightarrow 0$) while maintaining utility ($\text{LTCR} = 1.0, \text{FPR} = 0.0$) under bounded execution latency.

3. THREAT MODEL

3.1 Attacker Profile & In-Scope Vectors

The adversary is an external entity capable of manipulating untrusted text consumed by the agent. The adversary cannot modify model weights, system prompts, gateway source code, or the

underlying host OS. We consider five primary operational attack categories ($N = 100$ scenarios each):

- **T1: Unauthorized Tool Execution:** Coercing an agent into invoking tools outside its operational mandate (e.g., a support agent calling `execute_shell_command`).
- **T2: Privilege Escalation:** Forcing an agent to execute administrative functions (e.g., `update_customer_role`).
- **T3: Prompt-Driven Restricted Tool Execution:** Inducing high-risk destructive actions under an authorized tool (e.g., calling `file_writer` to overwrite `/etc/hosts`).
- **T4: Parameter Manipulation:** Injecting SQL syntax (`' OR '1'='1'`), shell chaining operators (`&&, |`), or directory traversal paths (`../../../../`) into permitted arguments.
- **T5: Excessive Invocations / Volumetric DoS:** Forcing automated recursive loops to exhaust API quotas or scrape data.

Separately, an **Adversarial Perturbation Suite** ($N = 500$) evaluates evasion transformations: Base64 encoding, URL percent-encoding, SQL comment fragmentation, case alternation, and stacked SQL logic.

3.2 Security Boundary & Assumptions

- **Trusted Base:** The reference monitor code, policy rulesets, RBAC configurations, SQLite state counters, and host operating system are trusted.
- **Untrusted Base:** The LLM reasoning engine, generated token sequences, and external environment observations are untrusted.
- **Out of Scope:**
 1. Direct host compromise or kernel exploitation.
 2. Uninstrumented tool execution: complete mediation is enforced *strictly for tool calls routed through the gateway interface*. Arbitrary code execution within an unconstrained bash tool that bypasses parameter inspection falls outside this boundary.
 3. Syntactically authorized actions with malicious intent (e.g., an agent authorized to delete temporary files being tricked into deleting a valid temporary file).

4. SECURITY BOUNDARY AND ARCHITECTURE

PromptAegis is implemented as an inline, application-level reference monitor. In conformance with `backend/governance/interceptor.py`, proposed actions are processed sequentially as depicted in Figure 1.

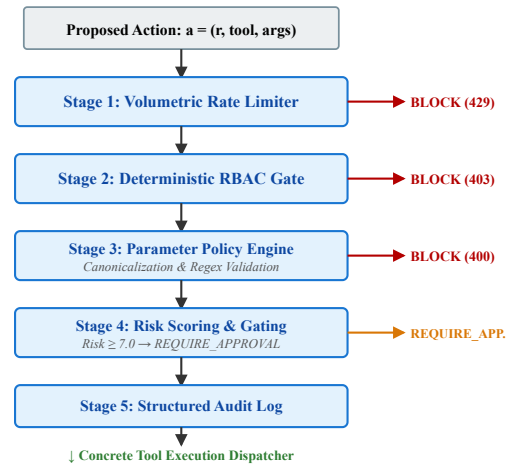


Figure 1: Sequential five-stage post-generation governance pipeline.

4.1 Complete Mediation Scope

PromptAegis provides application-level complete mediation strictly for structured tool invocations explicitly dispatched to its interface. Crucially, this operational boundary entails three explicit security constraints:

1. **Routed Tool Calls:** Every structured tool invocation dispatched to the gateway interceptor is deterministically evaluated against rate limits, RBAC permissions, and parameter policies prior to execution dispatch.
2. **Sub-Shell and Interpreter Boundary:** The gateway operates above the operating system syscall interface; if an agent is authorized to execute an unconstrained shell or code interpreter (e.g., `bash`, `python`), arbitrary commands executed internally within that sub-environment bypass parameter-level regex inspection.
3. **Semantic Malice within Authorized Parameters:** Purely syntactic policy evaluation cannot detect semantically hostile actions that utilize valid syntax and authorized parameters (e.g., an authorized customer query retrieving unauthorized sensitive records).

5. DEFENSIVE MECHANISMS

5.1 Stage 1: Volumetric Fixed Tumbling-Window Rate Limiter

Implemented in `backend/governance/rate_limiter.py`, Stage 1 partitions time into fixed 60-second intervals:

$$\text{BucketID}(t) = \left\lfloor \frac{t}{60.0} \right\rfloor$$

For principal p and tool τ with quota L , invocations are atomically counted in an SQLite store. Invocations where count exceeds L within the current 60s bucket are rejected with HTTP 429.

5.2 Stage 2: Deterministic RBAC Enforcement

Stage 2 validates the role-to-tool permission relation $\mathcal{P} \subseteq \mathcal{R} \times \mathcal{T}$ via `backend/governance/permission_engine.py`. If $(\tau, \tau) \notin \mathcal{P}$, execution is denied immediately without parameter processing.

5.3 Stage 3: Parameter Canonicalization & Policy Inspection

Stage 3 in `backend/governance/policy_engine.py` first canonicalizes string arguments:

- **Keyword-Agnostic Base64 Decoding (`_safe_b64_decode`):** Identifies Base64 candidates (`^[A-Za-z0-9+/=]{8,}$`). If

decoding yields valid UTF-8 with $\geq 85\%$ printable ASCII characters, the parameter is unmasked in-place.

- **Bounded URL Percent-Unquoting:** Applies a bounded 2-pass `urllib.parse.unquote`.
- **SQL Comment Stripping:** Removes `-- ... \n` and `/* ... */` fragments.

Canonicalized arguments are evaluated against compiled regular expressions bound to specific tool IDs.

5.4 Stage 4: Risk Scoring & Approval Gating

Stage 4 computes a composite risk score based on tool severity, argument payloads, and historical frequency. If `requires_approval` is enabled and the calculated score satisfies $\text{risk_score} \geq 7.0$, the gateway issues `REQUIRE_APPROVAL`, pausing dispatch.

5.5 Stage 5: Structured Audit Logging

Stage 5 persists structured audit records with execution timestamps, decision outcomes, latency measurements, and matched policy identifiers.

6. EXPERIMENTAL METHODOLOGY & BENCHMARK

6.1 Primary Benchmark Composition

The primary benchmark consists of $N = 600$ scenarios:

- **Synthetic Attack Corpus ($N = 500$):** 100 synthetic scenarios across each of the 5 primary threat categories (T1: Unauthorized Tool, T2: Privilege Escalation, T3: Restricted Execution, T4: Parameter Manipulation, T5: Excessive Calls), designed to systematically evaluate rule coverage against canonical injection patterns.
- **Benign Operational Corpus ($N = 100$):** 100 realistic operational commands reflecting standard software development and customer support tasks (e.g., `git status`, `search_customer`, `read_log_file`). Scenario `LEG-098` tests parameter precision by updating an authorized customer record where a string argument legitimately includes "password".

6.2 Deterministic Research Clock

To ensure bit-for-bit reproducibility, experiments run under `ResearchExperimentClock` (`experiments/research_clock.py`). Virtual time advances deterministically by $\Delta t = 70.0$ s under normalized evaluation (guaranteeing 0 rate-limiter events) and by $\Delta t = 0.05$ s under burst stress evaluation.

Table 1: Benchmark results across 6 configurations (3,000 attack evaluations, 3,600 total scenario decisions). Paired latency overhead reflects medians of paired per-scenario differences ($N = 500$ pairs); unvalidated paired statistics for RBAC, Policy, and Hardened are omitted.

Configuration	Arrival Regime (Δt)	Blocked ($N_{\text{block}}/500$)	ASR	LTCR	FPR	Median P50	Paired Overhead
Baseline (Unprotected)	Independent	0/500	100.0%	100.0% (100/100)	0.0% (0/100)	7.35 ms	Reference (0.00 ms)
RBAC Only	Independent	300/500	40.0%	100.0% (100/100)	0.0% (0/100)	8.09 ms	--
Policy Only	Independent	335/500	33.0%	100.0% (100/100)	0.0% (0/100)	8.99 ms	--
Full Normalized	$\Delta t = 70.0$ s	350/500	30.0%	100.0% (100/100)	0.0% (0/100)	19.08 ms	+11.27 ms
Full Burst	$\Delta t = 0.05$ s	370/500	26.0%	100.0% (100/100)	0.0% (0/100)	20.43 ms	+13.38 ms
Hardened Governance	$\Delta t = 0.05$ s	366/500	26.8%	100.0% (100/100)	0.0% (0/100)	24.21 ms	--

7. MECHANISM-ISOLATION ABLATION STUDY

Evaluating six configurations eliminates rate-limiter confounding:

1. **Isolated Semantic Performance:** Under `Full Normalized`, exactly 0 rate-limiting blocks occur. Semantic controls alone

achieve a 30.0% ASR (350/500 blocked). RBAC neutralizes Categories T1--T3 (300 blocks), while Policy catches 50 parameter injections (T4).

2. **Compound Burst Regime:** Under `Full Burst`, high-frequency traffic trips the rate limiter, reducing ASR to 26.0%. This

reduction is a compound operational effect resulting from traffic density, not a semantic gain.

8. CALIBRATED GATEWAY EVALUATION

We evaluate a calibrated gateway configuration featuring refined argument schemas and tool-scoped parameter fallback rules. The calibrated parameter rules (SQL injection regex on `query` arguments; restricted field guard scoped to `update_customer`) were designed and evaluated on the same $N = 600$ canonical benchmark corpus; no held-out evaluation split was employed.

- **Calibrated ASR:** 20.4% (102/500 allowed, 398/500 blocked; the full gateway stack, including rate limiting, is active in this configuration).
- **Utility Preservation:** 100.0% LTCR (100/100 completed), 0.0% FPR (0/100 blocked).
- **Resolution of LEG-098:** Scoping parameter rules strictly to `update_customer` prevents false rejections of legitimate strings containing "password".
- **Latency Profile:** Median calibrated processing time is 23.73 ms (P50).

9. ADVERSARIAL ROBUSTNESS EVALUATION

We evaluate 500 adversarial mutation instances structured as 5 perturbation transformation classes applied to 100 shared base attack templates ($5 \times 100 = 500$ instances). This clustered design evaluates evasion robustness across transformation types rather than 500 independent samples. Each instance is evaluated under Standard and Hardened configurations across two operational regimes, yielding 2,000 execution-level evaluations in total.

9.1 Regime A: Mechanism-Isolated Policy Evaluation

Isolates pure regex and canonicalization recall across 1,000 execution evaluations, reported in Table 2.

Table 2: Regime A adversarial results isolating semantic inspection (1,000 execution evaluations).

Mutation Class	Instances	Std Block	Hard Block	Delta	Std ASR
<code>case_alt.</code>	100	50.0%	33.0%	-17.0%	50.0%
<code>comment_frag.</code>	100	50.0%	33.0%	-17.0%	50.0%
<code>advanced_sql</code>	100	50.0%	33.0%	-17.0%	50.0%
<code>url_encoding</code>	100	50.0%	33.0%	-17.0%	50.0%
base64_obf.	100	0.0%	50.0%	+50.0%	100.0%
Overall	500	40.0%	36.4%	-3.6%	60.0%

9.2 Regime B: Compound Burst Gateway Evaluation

Evaluates the full runtime gateway under rapid arrival across 1,000 execution evaluations, reported in Table 3.

Table 3: Regime B compound burst results (1,000 execution evaluations). ASR is 24.8% (Std) and 25.6% (Hard).

Mutation Class	Std Burst	Hard Burst	Delta	Attribution (Std)
<code>case_alt.</code>	76.0%	74.0%	-2.0%	Policy: 4, RateLim: 72
<code>comment_frag.</code>	76.0%	74.0%	-2.0%	Policy: 4, RateLim: 72
<code>advanced_sql</code>	76.0%	74.0%	-2.0%	Policy: 4, RateLim: 72
<code>url_encoding</code>	76.0%	74.0%	-2.0%	Policy: 4, RateLim: 72
<code>base64_obf.</code>	72.0%	76.0%	+4.0%	Policy: 0, RateLim: 72
Overall	75.2%	74.4%	-0.8%	RateLim: 360, Pol: 16

9.3 Attribution and Non-Monotonicity Findings

- **Base64 Canonicalization Gain:** Standard regex exhibits 0.0% detection on Base64 payloads; keyword-agnostic canonicalization increases block rate to 50.0% (50/100).
- **Non-Monotonic Rule Trade-off:** Stripping comments alters string character offsets, degrading secondary regex trigger rates in multi-stage payloads (50% $\rightarrow$ 33%). Overall isolated block rate decreases from 40.0% to 36.4%.
- **Rate-Limiter Dominance in Burst:** In Regime B, 95.74% (360/376) of blocked attacks in the standard pipeline are enforced by the rate limiter. Burst results reflect volumetric throttling rather than semantic normalization.

10. RATE-LIMIT STRESS EVALUATION

Evaluating the fixed 60-second tumbling window across quotas $L \in \{5, 20, 100\}$ confirms exact saturation:

- Limit 5: Requests 1--5 allowed; Request #6 blocked (429 Too Many Requests) at $t = 5.0$ s.
- Limit 20: Request #21 blocked at $t = 20.0$ s.
- Limit 100: Request #101 blocked at $t = 100.0$ s.
- Reset: Advancing virtual time across the tumbling boundary to $t = 61.0$ s resets the counter, allowing Request #7.

11. EXPLORATORY CLOSED-LOOP LLM AGENT PILOT

An exploratory evaluation ($N = 20$) integrated PromptAegis into a multi-turn ReAct agent powered by `openai/gpt-oss-120b`:

- **Compromise Induction Rate:** The LLM succumbed to prompt injection in 40.0% (4/10) of attack episodes, generating unauthorized tool invocations.
- **Conditional Interception Rate:** PromptAegis intercepted 75.0% (3/4) of these compromised tool calls post-generation.
- **End-to-End System Breach Rate:** Only 10.0% (1/10) of attacks resulted in unintercepted tool execution in the target environment.
- **Benign Completion (LTCR):** 100.0% (10/10) of benign tasks completed.
- **Latency Accounting:** Mean model inference latency was 755.83 ms; mean gateway inspection latency was 161.11 ms (prompt-level ratio 21.32%). During active tool governance cycles, gateway overhead was 230.16 ms against 723.74 ms execution (tool-level ratio 31.80%).
- **Scope Note:** This pilot demonstrates operational feasibility in a live loop but is underpowered for general multi-model statistical claims.

12. STATISTICAL ANALYSIS

- **McNemar Tests:** Pairwise Edwards continuity-corrected χ^2 tests against the baseline ($N = 500$ pairs):
 - Normalized vs Baseline: $\chi^2 = 348.0029, p = 1.15 \times 10^{-77}$
 - RBAC vs Baseline: $\chi^2 = 298.0033, p = 8.97 \times 10^{-67}$
 - Policy vs Baseline: $\chi^2 = 333.0030, p = 2.13 \times 10^{-74}$
- Each tested treatment differed significantly from the unprotected baseline under the paired McNemar analysis ($p < 10^{-66}$).

- **Paired Latency Tests:** Wilcoxon signed-rank tests reject the null hypothesis of zero latency overhead ($p < 10^{-80}$). Documented paired overheads (+11.27 ms normalized, +13.38 ms burst) reflect medians of paired per-scenario differences ($d_i = L_{\text{treatment},i} - L_{\text{baseline},i}$).
- **Bootstrap Confidence Intervals:** 10,000 bootstrap resamples yielded 95% confidence intervals: Full Normalized ASR [26.0%, 34.0%]; RBAC ASR [35.8%, 44.2%]; Policy ASR [29.0%, 37.2%].

13. SECURITY--UTILITY--LATENCY TRADE-OFFS

Evaluating security, utility, and execution latency concurrently:

- **Security Reduction:** Attack success rate decreases from 100.0% to 30.0% under normalized semantic governance and to 20.4% under calibrated configuration.
- **Utility Preservation:** All configurations achieved 100.0% LTCR and 0.0% FPR on the benign benchmark.
- **Measured Latency Cost:** The median paired overhead is +11.27 ms under normalized traffic; in the closed-loop pilot, gateway inspection averaged 161.11 ms relative to 755.83 ms for model generation, corresponding to a 21.32% prompt-level overhead ratio.

14. FAILURE MODES AND NEGATIVE FINDINGS

1. **Semantic Parameter Smuggling:** In the closed-loop pilot, an injection induced the agent to call an authorized tool

Table 4: Architectural comparison of LLM safety and execution governance approaches.

Framework / Paradigm	Enforcement Point	Primary Mechanism	Determinism	Tool Mediation Scope
System Prompt Instructions [8]	Prompt Input	Natural language instructions	Probabilistic	Unmediated
RLHF Alignment [7]	Model Weights	Preference fine-tuning	Probabilistic	Unmediated
Llama Guard [9]	Token Stream	Secondary 7B LLM classifier	Probabilistic	Prompt/response safety classification
NeMo Guardrails [10]	Input/Output Dialog	Colang programmable dialog flows	Hybrid	Programmable action gating
Guardrails AI [14]	Output Parser	RAIL specifications & validator pipelines	Rule / validator-dependent	Input/output validation & structural constraints
OS Sandboxes (Docker/Seccomp) [5, 15]	Kernel Syscall	Cgroups [15, 17], namespaces [15, 16], seccomp [18]	Deterministic	Process isolation & selective syscall filtering
PromptAegis (Ours)	Tool Dispatch Interface	RBAC, Normalization, Policy, Rate Limit	Deterministic	Complete routed tool-call mediation

16. RELATED WORK

Prior research in agent security largely emphasizes input filtering [8, 9] or conversational flow management [10]. While Llama Guard [9] inspects conversational tokens, secondary classifiers introduce additional model inference passes. NeMo Guardrails [10] provides programmable dialogue and execution-flow constraints, whereas PromptAegis evaluates concrete structured tool calls using explicit RBAC matrices, parameter canonicalization, and rate limits. PromptAegis adapts classical reference monitor principles [12, 13] to the structured tool-dispatch path.

17. LIMITATIONS

- **Syntactic Bound:** PromptAegis inspects structured parameters via schemas and regexes; it does not perform semantic reasoning.
- **Enforcement Path Dependence:** The gateway protects only tools routed through its interface.

(`search_customer`) with valid syntax (`query: "all_customer_emails"`). Because the command was syntactically benign and permitted under the active role, the gateway allowed it. Purely syntactic reference monitors cannot infer semantic intent.

2. **Non-Monotonic Normalization:** Stripping comments altered string offsets, degrading secondary regex triggers and reducing isolated block rate from 40.0% to 36.4%.
3. **Burst Rate-Limiter Attribution:** 95.74% of burst blocks are caused by rate limiting, not semantic inspection.
4. **Encoding Depth Limitations:** Single-pass Base64 decoding unmasked 50.0% of payloads, but multi-nested or custom encodings evaded detection.

15. THREATS TO VALIDITY

- **Internal Validity:** Benchmark results depend on synthetic attack templates.
- **External Validity:** Closed-loop testing evaluated a single model (gpt-oss-120b). Generalization across diverse model families remains unmeasured.
- **Gateway Bypass Scope:** Direct OS execution within an unconstrained shell bypasses parameter-level inspection.
- **Tumbling-Window Boundary Effect:** Fixed tumbling windows permit boundary clustering (2L calls within 2 seconds across the 60s reset).

- **Exploratory Pilot Scale:** The closed-loop pilot ($N = 20$) is an initial proof-of-concept.
- **In-Sample Calibration:** The calibrated gateway configuration was evaluated on the same benchmark corpus used to design the parameter rules; out-of-sample generalization of the calibrated configuration has not been established.

18. CONCLUSION

Securing autonomous LLM agents requires decoupling cognitive reasoning from concrete downstream execution. The PromptAegis gateway demonstrates that an application-level, deterministic reference monitor can reduce attack success rates from 100.0% to 30.0% under normalized semantic governance and to 20.4% under calibrated configuration on the evaluated benchmark, while preserving 100.0% benign task completion with a median paired overhead of +11.27 ms. However, protection remains bounded

by policy expressiveness, cannot prevent semantic parameter smuggling within authorized domains, and does not alter the underlying vulnerability of the reasoning model.

19. REPRODUCIBILITY & ARTIFACT AVAILABILITY

All evaluation datasets, configuration files, and execution scripts are frozen to ensure exact reproducibility:

- **Repository:**

https://github.com/Shriram28Baskar/PromptAegis_RI

20. REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [2] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2023.
- [3] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [4] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proc. IEEE*, vol. 63, no. 9, pp. 1278--1308, 1975.
- [5] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux Journal*, vol. 2014, no. 239, 2014.
- [6] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proc. ACM Workshop Offensive Technol. (WOOT)*, 2023.
- [7] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., "Training language models to follow instructions with human feedback," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2022.
- [8] F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques For Language Models," in *Proc. NeurIPS Workshop on ML Safety*, 2022 (arXiv:2211.09527).
- [9] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations," *arXiv preprint arXiv:2312.06674*, 2023.
- [10] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications," *arXiv preprint arXiv:2310.10501*, 2023.
- [11] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and Transferable Adversarial Attacks on Aligned Language Models," *arXiv preprint arXiv:2307.15043*, 2023.
- [12] J. P. Anderson, "Computer Security Technology Planning Study," Tech. Rep. ESD-TR-73-51, Air Force Electronic Systems Division, 1972.
- [13] F. B. Schneider, "Enforceable Security Policies," *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 1, pp. 30--50, 2000.
- [14] Guardrails AI, "Guardrails: Adding Guardrails to Large Language Models," *GitHub Repository*, <https://github.com/guardrails-ai/guardrails>, 2023.
- [15] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press, San Francisco, CA, 2010.
- [16] M. Kerrisk, "namespaces(7) --- overview of Linux namespaces," in *Linux Programmer's Manual*, Linux Man-Pages Project, 2023.
- [17] M. Kerrisk, "cgroups(7) --- Linux control groups," in *Linux Programmer's Manual*, Linux Man-Pages Project, 2023.
- [18] M. Kerrisk, "seccomp(2) --- operate on Secure Computing state of the process," in *Linux Programmer's Manual*, Linux Man-Pages Project, 2023.

- **Frozen Baseline Commit:**

`f16fae2cffece5b032e47bb7dfc52220c2eae1fc`

- **Verification:** The test suite passes 5/5 assertions. Complete reproduction executes via:

```
python -m pytest tests/test_provenance_and_reproducibility.py
python experiments/run_all.py
```